30-Day Learning Plan: From Zero to Production React Native + Expo Apps

Who this plan is for

This plan is for **complete beginners** (students) who want to build real, production-quality mobile apps with **React Native** and **Expo**. You do not need any programming experience. You need:

- 4–5 hours per day, for 30 days.
- A computer (Windows, macOS or Linux) and a phone (Android or iPhone).
- Patience. Feeling confused is normal. Confusion means you are learning.

By Day 30 you will have built a new feature module inside this starter project (`expo-layered-crud`), with tests, and opened a pull request for it, the same way professional teams work.

How to use this plan

Your daily routine

1. **Warm up (10 min):** read yesterday's journal entry and the "Done when" list of yesterday.
2. **Learn:** read the links for the day. Do not try to memorise everything. Aim to understand the idea, then come back to the docs when you need them.
3. **Practice:** type every example yourself. Do not copy and paste. Your fingers learn too.
4. **Build / review:** do the exercises, then tick the "Done when" checklist.
5. **Push your work (10 min):** commit and push what you did today to your own GitHub repository (you create it on Day 2).
6. **Journal (10 min):** write your learning journal entry.

Keep a learning journal

Create a file called `JOURNAL.md` in your practice repository. Every day, write:

- **What I learned today** (3–5 bullets in your own words).
- **What confused me** (and whether I solved it).
- **One thing I want to ask my mentor.**

Writing things in your own words is the fastest way to find out what you really understand.

How to ask for help

Try for **20–30 minutes** on your own first. Then ask. A good question contains:

1. **What you are trying to do** ("I want the product list to sort by price").
2. **What you expected** and **what actually happened**.
3. **The exact error message** (copy the text, do not paraphrase) and the file name.
4. **What you already tried.**
5. A link to your code on GitHub (a commit or a branch).

A good question often answers itself while you write it. That is fine. That is a skill too.

Review / buffer days

Days **7, 14, 21 and 28** contain a review block. Days **14 and 21** are full review/buffer days: use them to catch up if you fall behind, redo exercises you skipped, and answer the weekly self-assessment questions at the end of this document. Falling behind by a day or two is normal. Skipping the review days is not a good idea.

Prerequisites and setup checklist

Do this before Day 1 (or during the first hour of Day 1).

- [ ] **Node.js LTS (Long-Term Support)** installed. Check with `node -v` in a terminal. This project needs version 20.19.4 or newer.
- [ ] **VS Code (Visual Studio Code)** installed, with these extensions:
  - [ ] ESLint (finds problems in your code)
  - [ ] Prettier (formats your code automatically)
  - [ ] Tailwind CSS IntelliSense (autocomplete for `className` styles)
- [ ] **Git** installed. Check with `git --version`.
- [ ] A **GitHub** account (free) at github.com.
- [ ] **Expo Go** app installed on your phone from the App Store / Google Play. It must support **Expo SDK (Software Development Kit) 58**, which is the version this project uses.
- [ ] Your phone and computer are on the **same Wi-Fi network** (needed to open the app in Expo Go).

The 30 days at a glance

Day	Topic	Phase
1	How apps and the web work + terminal basics	1. Foundations
2	Git and GitHub	1. Foundations
3	JavaScript I: values, variables, functions	1. Foundations

Day	Topic	Phase
4	JavaScript II: arrays, objects, modules, array methods	1. Foundations
5	JavaScript III: async code, fetch, errors, closures	1. Foundations
6	TypeScript I: types, interfaces, unions, narrowing	1. Foundations
7	TypeScript II: generics, `as const`, `satisfies`, utility types + Week 1 review	1. Foundations
8	React I: components, JSX, props, children	2. React
9	React II: state, events, conditional rendering, lists	2. React
10	React III: forms, controlled inputs, effects (and when not to use them)	2. React
11	React IV: lifting state, custom hooks, rules of hooks	2. React
12	Thinking in React + React with TypeScript	2. React
13	Expo setup, Expo Go and React Native core components	3. React Native + Expo
14	Week 2 review and buffer day	3. React Native + Expo
15	Flexbox layout, NativeWind styling, theme tokens, dark mode	3. React Native + Expo
16	Lists: FlatList, FlashList, images	3. React Native + Expo
17	Expo Router I: file-based routes, layouts, Stack, Tabs	3. React Native + Expo
18	Expo Router II: dynamic routes, params, typed routes	3. React Native + Expo
19	Safe areas, keyboard, platform differences, UI primitives	3. React Native + Expo
20	The HTTP client, error model, endpoints, environment variables	4. Data layer
21	Week 3 review and buffer day	4. Data layer
22	zod validation and the service layer	4. Data layer
23	React Query I: queries, query keys, `queryOptions`, screen states	4. Data layer
24	React Query II: infinite queries, mutations, cache updates, optimistic updates	4. Data layer
25	Forms with react-hook-form + zod	4. Data layer
26	Modular architecture: modules, features, ESLint boundaries	5. Architecture and shipping
27	Testing with Jest + Conventional Commits and pull requests	5. Architecture and shipping
28	Debugging, performance, accessibility + Week 4 review	5. Architecture and shipping
29	Development builds, EAS Build / Submit / Update + capstone part 1	5. Architecture and shipping
30	Capstone part 2: finish the "users" module and open a pull request	5. Architecture and shipping

Phase 1 — Foundations (Days 1–7)

Day 1 — How Apps and the Web Work + Terminal Basics

Goal: Understand what happens when an app shows data from the internet, and get comfortable typing commands in a terminal.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Client** = the app on your phone (or the browser). **Server** = a computer somewhere that stores data and answers requests.
- **HTTP (HyperText Transfer Protocol)** = the language clients and servers use. A request has a **method** (`GET` read, `POST` create, `PUT` / `PATCH` update, `DELETE` remove), a **URL (Uniform Resource Locator)**, headers and an optional body.
- A response has a **status code**: `200` OK, `201` Created, `400` Bad Request, `401` Unauthorized, `404` Not Found, `500` Server Error.
- **JSON (JavaScript Object Notation)** = the text format most APIs use to send data: `{ "id": 1, "title": "Phone" }`.
- **API (Application Programming Interface)** = a set of URLs a server offers so apps can read and change data. This project uses the free **DummyJSON** API.
- **Terminal basics:** `pwd` (where am I), `ls` (list files; `dir` on Windows), `cd folder` (go into), `cd ..` (go up), `mkdir` (make folder), `touch` / `code .` (create file / open VS Code), arrow-up for history, `Ctrl + C` to stop a running program.

Read / watch

- javascript.info — read "An Introduction" (first chapters only)
- DummyJSON — Products docs

Practice

1. Open `https://dummyjson.com/products/1` in your browser. Identify 5 fields and their types (text, number, list, object).
2. Open `https://dummyjson.com/products?limit=3&skip=0`. What do `limit` and `skip` do? Try `skip=3`.
3. Open `https://dummyjson.com/products/999999`. What status code and message do you get?
4. In the terminal: create a folder `rn-learning`, go into it, create `notes.txt`, list the files, go back up.
5. Draw (on paper) the journey: "user taps Products tab → app sends GET request → server answers JSON → app shows a list".

In this repo

- The base URL of the API lives in `src/shared/config/env.ts` (`https://dummyjson.com`).
- All the URLs the app calls are listed in `src/modules/catalog/data/catalog.endpoints.ts`. Open it and match each one to a DummyJSON docs section.

Done when

- [ ] I can explain client, server, HTTP, API and JSON to a friend in one sentence each.
- [ ] I know what 200, 201, 404 and 500 mean.
- [ ] I can move around folders and create files using only the terminal.
- [ ] I started `JOURNAL.md`.

Day 2 — Git and GitHub

Goal: Save your work history with Git and share it on GitHub.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Git** = a tool that records snapshots ("commits") of your project, so you can go back in time.
- **Repository (repo)** = a project folder tracked by Git. **GitHub** = a website that hosts repos online.
- Core commands: `git init`, `git status`, `git add`, `git commit -m "message"`, `git log --oneline`, `git diff`.
- **Branch** = a separate line of work (`git switch -c my-branch`). `main` is the default branch.
- **Remote** = the online copy. `git clone`, `git push`, `git pull`.
- **Pull request (PR)** = a request to merge your branch into `main`, where others review your code.
- `.gitignore` = files Git should never track (like `node_modules`, `.env`).

Read / watch

- Pro Git book — chapters 1 (Getting Started) and 2 (Git Basics); skim chapter 3 (Branching)

Practice

1. Configure Git: `git config --global user.name "Your Name"` and `user.email`.
2. Turn `rn-learning` into a repo, add `JOURNAL.md`, make your first commit.
3. Create a new empty repo on GitHub called `rn-learning`, connect it (`git remote add origin ...`) and push.
4. Create a branch `day-2-practice`, change a file, commit, push, and open a pull request on GitHub. Merge it yourself.
5. **Clone this starter project** (`expo-layered-crud`) to your computer. Run `git log --oneline` and read the last 5 commit messages.

In this repo

- Look at `.gitignore` — which files are never committed?
- Look at `.github/pull_request_template.md` — this is the checklist every PR in this project must fill in.

Done when

- [ ] My `rn-learning` repo is on GitHub with at least 3 commits.
- [ ] I opened and merged one pull request.
- [ ] I can explain the difference between commit, push, branch and pull request.
- [ ] I cloned this starter project.

Day 3 — JavaScript I: Values, Variables and Functions

Goal: Write small JavaScript programs using variables, conditions, loops and functions.

Time split (≈4–5 h): Learn 2 h · Practice 2.5 h · Build/review 0.5 h

Learn

- Run JavaScript with Node.js: create `day3.js`, run `node day3.js`.
- `const` (cannot be reassigned, use by default) vs `let` (can change). Never use `var`.
- Types: string, number, boolean, `null`, `undefined`, object, array.
- `===` (strict equal; always use this) vs `==` (avoid).
- Template strings: `` `Hello ${name}` ``.
- `if / else`, ternary `cond ? a : b`, `&&`, `||`, `??` (nullish coalescing: use the right side only if the left is `null` / `undefined`), optional chaining `user?.address?.city`.
- Loops: `for`, `for...of`.
- Functions: function declarations, parameters, default parameters, return values.
- **Arrow functions:** `const add = (a, b) => a + b;`

Read / watch

- javascript.info — Part 1, "JavaScript Fundamentals" (variables to arrow functions)
- MDN (Mozilla Developer Network) — Dynamic scripting with JavaScript

Practice

1. Write `formatPrice(amount)` that returns `"$12.50"` for `12.5`.
2. Write `stockLabel(stock)` that returns `"Out of stock"` for 0, `"Low stock"` for 1–10, `"In stock"` otherwise.
3. Rewrite both as arrow functions.
4. Write `greet(name = "friend")` with a default parameter.
5. Use `??` to show `"No brand"` when `product.brand` is `undefined`.

In this repo

- Open `src/shared/lib/format.ts` — real formatting helpers. Which ones can you read already?
- Open `src/modules/catalog/components/stock-badge.tsx` and find the logic similar to your `stockLabel`.

Done when

- [ ] I know when to use `const` and when to use `let`.
- [ ] I can write a function both as a declaration and as an arrow function.
- [ ] I can explain the difference between `||` and `??`.
- [ ] My exercises are pushed to GitHub.

Day 4 — JavaScript II: Arrays, Objects, Modules and Array Methods

Goal: Transform lists of data the way real apps do.

Time split (≈4–5 h): Learn 2 h · Practice 2.5 h · Build/review 0.5 h

Learn

- Objects `{ key: value }`, arrays `[1, 2, 3]`, nested data.
- **Destructuring:** `const { title, price } = product;` and `const [first, second] = list;`
- **Spread / rest:** `{ ...product, price: 10 }` copies an object and changes one field (this is how we "update" data without mutating it). `[...list, newItem]` adds to a copy.
- **Immutability:** create new arrays/objects instead of changing old ones. React depends on this.
- Array methods: `map` (transform each item), `filter` (keep some), `find` (first match), `some` / `every`, `reduce` (combine into one value), `sort` (careful: it mutates; use `[...list].sort()` or `toSorted()`).
- **Modules:** `export` / `import`, named exports vs `export default`.

Read / watch

- javascript.info — "Objects: the basics", "Data types" (arrays, array methods, destructuring), and "Modules"

Practice

1. Fetch `https://dummyjson.com/products?limit=10` in the browser, copy the `products` array into `products.js`.
2. Using `map`, make a list of titles.
3. Using `filter`, keep products cheaper than 20.
4. Using `find`, get the product with `id === 3`.
5. Using `reduce`, compute the total stock.
6. Sort by price (low to high) without changing the original array.
7. Write `updatePrice(product, newPrice)` that returns a **new** object using spread.
8. Split your code into `products.js` (export data) and `main.js` (import and use it).

In this repo

- `src/modules/catalog/index.ts` is a **module's public API**: it only re-exports things. Read the comment at the top.
- Search the project for `.map(` and `.filter(` (VS Code: `Ctrl/Cmd + Shift + F`) and read 3 examples.

Done when

- [ ] I can use `map`, `filter`, `find` and `reduce` without looking them up.
- [ ] I can update an object or array immutably with spread.
- [ ] I can split code into files with `import` / `export`.

Day 5 — JavaScript III: Async Code, fetch, Errors and Closures

Goal: Load real data from an API and handle failures.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- JavaScript runs one thing at a time; slow work (network) is **asynchronous**.
- **Promise** = a value that will arrive later (or fail). States: pending, fulfilled, rejected.
- `async` / `await` makes async code read like normal code.
- `fetch(url)` returns a Promise of a Response; `await res.json()` reads the body. **`fetch` does not throw on 404 or 500** — you must check `res.ok`.
- Errors: `throw new Error("...")`, `try { } catch (error) { } finally { }`.
- `Promise.all` to run requests in parallel.
- **Closure** = a function that remembers variables from where it was created. Used by debounce, counters, event handlers and React hooks.

Read / watch

- javascript.info — "Promises, async/await", "Error handling", "Variable scope, closure", and Part 3 "Network requests: Fetch"

Practice

1. Write `async function getProduct(id)` that fetches `https://dummyjson.com/products/${id}` and returns the JSON.
2. Make it throw an `Error` with the status code when `res.ok` is false. Test with id `999999`.
3. Call it inside `try/catch` and print a friendly message on failure.
4. Load products 1, 2 and 3 in parallel with `Promise.all`.
5. Write `makeCounter()` that returns a function; each call returns the next number (closure).
6. **Stretch:** write a simple `debounce(fn, ms)` using `setTimeout` and a closure.

In this repo

- `src/shared/api/http-client.ts` is the **only** place in the app that calls `fetch`. Read it slowly: find `async`, `await`, `try/catch`, the `res.ok` check and the timeout.
- `src/shared/hooks/use-debounced-value.ts` is the React version of your debounce.

Done when

- [ ] I can fetch JSON with `async/await` and handle a 404.
- [ ] I can explain why `fetch` needs a `res.ok` check.
- [ ] I can explain what a closure is with my counter example.

Day 6 — TypeScript I: Types, Interfaces, Unions and Narrowing

Goal: Describe the shape of your data with TypeScript so mistakes are caught before the app runs.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **TypeScript (TS)** = JavaScript + types. The **compiler** (`tsc`) checks your code; types disappear when the app runs.
- Basic types: `string` , `number` , `boolean` , arrays `string[]` , objects.
- **Type inference:** TS often figures out the type for you; do not annotate everything.
- `interface Product { id: number; title: string }` vs `type Product = { ... }` . Both describe objects; `type` can also describe unions and other shapes. Pick one style per project (see `docs/CONVENTIONS.md`).
- **Optional fields:** `brand?: string` .
- **Union types:** `string | number` . **Literal types:** `type Order = 'asc' | 'desc'` .
- **Typing functions:** `function add(a: number, b: number): number` , and typing arrow functions.
- **Narrowing:** `typeof x === 'string'` , `if (value)` , `'key' in obj` , `instanceof` — TS understands checks and narrows the type inside the `if` .
- `unknown` (safe, must narrow before use) vs `any` (turns checking off; avoid).

Read / watch

- TypeScript Handbook — Introduction — read "The Basics", "Everyday Types" and "Narrowing"

Practice

1. Run `npm init -y` and `npm install -D typescript` in your practice folder, create `day6.ts` , check it with `npx tsc --noEmit day6.ts` .
2. Write a `Product` type (id, title, price, stock, optional brand).
3. Retype yesterday's `getProduct(id: number): Promise<Product>` .
4. Write `type SortOrder = 'asc' | 'desc'` and a function that only accepts it. Try passing `'up'` and read the error.
5. Write `function describe(value: string | number)` that narrows with `typeof` .
6. Write `function getMessage(error: unknown): string` that narrows with `instanceof Error` .

In this repo

- `src/shared/api/api-error.ts` : `ApiErrorKind` is a union of literal types (`network` , `timeout` , `http` , `parse`). `isApiError` is a **type guard** used for narrowing. `getErrorMessage(error: unknown)` is exactly exercise 6.
- Run `npm install` then `npm run typecheck` in this repo. It should pass with no errors.

Done when

- [ ] I can write types for an API object with optional fields.
- [ ] I can use union and literal types.

- [ ] I can narrow `unknown` safely.
- [ ] `npm run typecheck` runs in this repo.

Day 7 — TypeScript II: Generics, `as const`, `satisfies`, Utility Types + Week 1 Review

Goal: Read the TypeScript you will see in real projects, then review the whole week.

Time split (≈4–5 h): Learn 1.5 h · Practice 1.5 h · Review 1.5 h

Learn

- **Generics** = types with parameters: `function first<T>(list: T[]): T | undefined`. `Promise<Product>`, `Array<string>` are generics too.
- `as const` makes values read-only and keeps literal types: `['products', 'list'] as const`.
- `satisfies` checks a value matches a type **without** losing its more precise type.
- **Utility types:** `Partial<T>` (all fields optional — great for "update" payloads), `Pick<T, 'a' | 'b'>`, `Omit<T, 'id'>` (great for "create" payloads), `Record<K, V>`, `ReturnType<typeof fn>`.
- Reading error messages: read the **last** line first, then go up.

Read / watch

- TypeScript Handbook — Generics
- TypeScript Handbook — skim "Object Types" and "Utility Types"

Practice

1. Write a generic `function last<T>(list: T[]): T | undefined`.
2. Write `type NewProduct = Omit<Product, 'id'>` and `type ProductUpdate = Partial<NewProduct>`.
3. Write a `routes` object with `as const` and see how the hover type changes in VS Code.
4. Use `satisfies Record<string, string>` on a config object and try adding a number value.
5. **Review:** redo one exercise from each day (1–6) without looking at your old code.
6. **Mini-project:** a TypeScript script that fetches 30 products, prints the 5 cheapest titles with formatted prices, and handles network errors.

In this repo

- `src/modules/catalog/data/product.keys.ts` uses `as const` on every query key.
- `src/shared/config/env.ts` ends with `as const`.
- `src/shared/api/http-client.ts` has `request<T = unknown>(...)` — a generic function.
- `src/modules/catalog/data/products.service.ts`: `update(id, payload: Partial<ProductPayload>)` uses `Partial`.

Done when

- [ ] I can read and write a simple generic function.
- [ ] I can explain `as const`, `satisfies`, `Partial`, `Pick` and `Omit`.
- [ ] I finished the mini-project and answered the Week 1 self-assessment questions.

Phase 2 — React Fundamentals (Days 8–12)

> For Days 8–12 you can practise React in the browser using the sandboxes linked inside react.dev, or directly in this repo's UI Sandbox screen once you reach Day 13. The concepts are the same in React Native; only the building blocks (`View` , `Text`) differ from the web (`div` , `p`).

Day 8 — React I: Components, JSX, Props and Children

Goal: Build a user interface out of small, reusable components.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Component** = a function that returns UI. Names start with a capital letter.
- **JSX (JavaScript XML)** = HTML-like syntax inside JavaScript. Use `{ }` to put JavaScript values inside.
- **Props** = inputs to a component, like function arguments. Props are read-only.
- **Composition** with `children`: a `Card` can wrap any content.
- One component per file, small and focused. Export, import, reuse.

Read / watch

- React — Learn — "Quick Start", then "Describing the UI": "Your First Component", "Importing and Exporting Components", "Writing Markup with JSX", "JavaScript in JSX with Curly Braces", "Passing Props to a Component"

Practice

1. Build `ProductCard` that receives `title`, `price` and `thumbnail` props.
2. Build `PriceTag` and use it inside `ProductCard` (composition).
3. Build a `Card` component that renders `children` and wrap `ProductCard` content in it.
4. Render 3 `ProductCard`s with different props.

In this repo

- `src/modules/catalog/features/product-list/product-card.tsx` — a real product card. Find its props.
- `src/shared/ui/card.tsx` — a `Card` primitive that uses composition (`Card`, `CardHeader`, `CardContent` ...).

Done when

- [ ] I can create a component, pass props to it and render it several times.
- [ ] I can explain what `children` is.
- [ ] I can read `product-card.tsx` and name its props.

Day 9 — React II: State, Events, Conditional Rendering and Lists

Goal: Make components interactive.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **State** with `useState`: memory that belongs to a component. Changing state re-renders the component.
- Never change state directly; call the setter with a new value (spread from Day 4!). Use the updater form `setCount(c => c + 1)` when the new value depends on the old one.
- **Events:** `onClick` on the web, `onPress` in React Native.
- **Conditional rendering:** `if` + early return, `cond ? <A/> : <B/>`, `cond && <A/>` (careful: `0 && ...` renders `0`).
- **Lists & keys:** `items.map(item => <Row key={item.id} ... />)`. Keys must be stable and unique — use ids, not the array index.

Read / watch

- React — Learn — "Adding Interactivity": "Responding to Events", "State: A Component's Memory", "Render and Commit", "Updating Objects in State", "Updating Arrays in State"; and "Conditional Rendering", "Rendering Lists"

Practice

1. A counter with + and − buttons that cannot go below 0.
2. A "favorite" toggle (heart filled / empty).
3. A product list with a "Show only in-stock" toggle (state + `filter`).
4. A "sort by price" button that switches between ascending and descending.
5. Show "No products" when the filtered list is empty.

In this repo

- Search for `useState` in `src/modules/catalog/features/product-list/product-list-screen.tsx` — what state does the list screen keep (hint: the search text)?
- `src/shared/components/screen-states.tsx` — Loading / Error / Empty components used for conditional rendering everywhere.

Done when

- [ ] I can add state to a component and update it from an event.
- [ ] I can render a list with correct keys.
- [ ] I can show an "empty" message conditionally.

Day 10 — React III: Controlled Inputs and Effects (and When Not to Use Them)

Goal: Handle user input and understand `useEffect` well enough to avoid its traps.

Time split (≈4–5 h): Learn 2.5 h · Practice 1.5 h · Build/review 1 h

Learn

- **Controlled input:** the input's value comes from state, and every change updates state.
- `useEffect` synchronises your component with something **outside** React (a timer, a subscription, a network connection). It runs after render.
- Dependency array: `[]` = run once after mount; `[a, b]` = run when `a` or `b` changes. Return a **cleanup** function.
- **When NOT to use an effect:**
  - To compute a value from props/state → just compute it during render.
  - To react to a user event → put the code in the event handler.
  - To fetch server data → in this project we use **React Query** (Day 23), not `useEffect` + `fetch`.
- React 19 Strict Mode runs effects twice in development to help you find missing cleanups.

Read / watch

- React — Learn — "Reacting to Input with State", "Choosing the State Structure", "Synchronizing with Effects", "You Might Not Need an Effect"

Practice

1. A search input (controlled) that filters a product list as you type.
2. Show "Typed X characters" — computed during render, **no** effect.
3. A clock that updates every second with `setInterval` in an effect, with cleanup.
4. Find the bug: write an effect that copies `props.items.length` into state, then delete it and compute it directly instead.

In this repo

- `src/shared/hooks/use-debounced-value.ts` uses `useEffect` with cleanup (`clearTimeout`) — a correct use of an effect.
- Notice that no screen in `src/modules` uses `useEffect` to fetch data. Why? (Answer on Day 23.)

Done when

- [ ] I can build a controlled text input.
- [ ] I can write an effect with a cleanup.
- [ ] I can name 3 cases where I should NOT use `useEffect`.

Day 11 — React IV: Lifting State, Custom Hooks and the Rules of Hooks

Goal: Share state between components and package logic into reusable hooks.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Lifting state up:** when two components need the same state, move it to their closest common parent and pass it down as props.
- **Single source of truth:** every piece of state has one owner.
- **Custom hooks:** functions starting with `use` that call other hooks. They share **logic**, not state.
- **Rules of hooks:** only call hooks at the top level of a component or custom hook — never inside `if`, loops or after an early `return`. ESLint checks this for you.
- `useMemo` / `useCallback` exist for performance; do not reach for them by default.

Read / watch

- React — Learn — "Sharing State Between Components", "Preserving and Resetting State", "Reusing Logic with Custom Hooks"

Practice

1. Build `SearchBar` + `ProductList` siblings; lift the search text into their parent.
2. Write a custom hook `useToggle(initial)` returning `[value, toggle]`.
3. Write `useDebouncedValue(value, delay)` yourself, then compare with the repo version.
4. Break the rules on purpose (call a hook inside an `if`) and read the ESLint error.

In this repo

- `src/shared/hooks/` — `use-debounced-value.ts`, `use-theme-colors.ts`, `use-value-while-focused.ts`. Read each and write one sentence about what it does.
- Every React Query hook in `src/modules/catalog/data/product.queries.ts` (`useProduct`, `useCategories` ...) is a custom hook.

Done when

- [ ] I can lift state to a parent and pass it down.
- [ ] I wrote two custom hooks.
- [ ] I can state the rules of hooks.

Day 12 — Thinking in React + React with TypeScript

Goal: Plan a screen like a professional, then type your components.

Time split (≈4–5 h): Learn 1.5 h · Practice 1 h · Build/review 2 h

Learn

- **Thinking in React:** 1) break the UI into a component tree, 2) build a static version, 3) find the minimal state, 4) decide where state lives, 5) add inverse data flow (callbacks).
- Typing props: `type ProductCardProps = { title: string; price: number; onPress?: () => void }`.
- Typing `children`: `React.ReactNode`. Typing events and state: `useState<string | null>(null)`.
- Prefer named function components with typed props over `React.FC`.

Read / watch

- Thinking in React
- React + TypeScript Cheatsheet — "Function Components", "Hooks", "Typing Component Props"

Practice

1. Follow "Thinking in React" for a **product catalog page**: draw the component tree on paper first.
2. Build it in TypeScript: search box, "in stock only" checkbox, list grouped by category.
3. Type every component's props. Remove all `any`.
4. Push to GitHub and write in your journal: "Which state did I have, and which component owns it?"

In this repo

- Open `src/modules/catalog/features/product-list/` and draw its component tree: `ProductListScreen` → search input, `ProductCard`, `ProductActionsSheet`, `ProductListSkeleton`.
- Look at the props type at the top of `src/modules/catalog/components/product-form.tsx`.

Done when

- [ ] I can draw a component tree for any screen and say where state lives.
- [ ] My catalog page is fully typed with no `any`.
- [ ] I can read the props types of components in this repo.

Phase 3 — React Native + Expo (Days 13–19)

Day 13 — Expo Setup, Expo Go and React Native Core Components

Goal: Run this project on your own phone and build screens with the core components.

Time split (≈4–5 h): Learn 1.5 h · Practice 2 h · Build/review 1.5 h

Learn

- **React Native (RN)** turns React components into real native views (not a web page).
- **Expo** = tools and libraries on top of React Native: a dev server, Expo Go, Expo Router, EAS (Expo Application Services) and many SDK modules.
- **Expo Go** = an app that runs your project during development by scanning a QR code. It only includes Expo's built-in native modules.
- **Metro** = the bundler that builds your JavaScript. **Fast Refresh** updates the app when you save.
- Core components: `View` (box), `Text` (all text must be inside `Text`), `Pressable` (tappable), `ScrollView` (scrollable content), `TextInput`, `Image` (this project uses **expo-image**).
- Read the major version of `expo` in `package.json` (58) and always use the docs for that version — Expo changes between releases.

Read / watch

- React Native — Getting Started — "Core Components and Native Components", "React Fundamentals"
- Expo Tutorial — Introduction
- Repo: Getting started

Practice

1. In this repo: `npm install`, then `npm start`. Scan the QR code with Expo Go (Android) or the Camera app (iPhone).
2. Visit every tab: Home, Products, Settings. Open the UI Sandbox from Settings.
3. Change a text in `src/modules/home/features/dashboard/home-screen.tsx`, save, and watch Fast Refresh. Undo it.
4. In your own practice project (`npx create-expo-app@latest`), build a profile screen with `View`, `Text`, `Image`, a `TextInput` and a `Pressable` that shows an alert.
5. Wrap long content in a `ScrollView`.

In this repo

- `package.json` — find `expo`, `react-native`, `react` versions and the `scripts`.
- `src/app/` holds routes only; screens live in `src/modules/<module>/features/<feature>/`.
- `src/modules/home/features/dashboard/home-screen.tsx` — the Home tab.

Done when

- [ ] This project runs on my phone in Expo Go.
- [ ] I can name 6 core components and what they do.
- [ ] I built a small screen with them in my practice project.

Day 14 — Week 2 Review and Buffer Day

Goal: Close the gaps from Days 8–13 before moving on.

Time split (≈4–5 h): Review 2 h · Practice 2 h · Journal & plan 1 h

Learn

- No new concepts. Revisit anything you marked "confused" in your journal.
- Re-read "You Might Not Need an Effect" — it is the most important React page for this project.

Read / watch

- React — Learn — any chapter you skipped
- React Native — Getting Started

Practice

1. Rebuild your Day 12 catalog page in React Native in your practice project (`View` / `Text` / `Pressable` instead of HTML).
2. Load real products from DummyJSON with `fetch` in an effect **once**, just to feel the pain: handle loading, error and empty states by hand. (On Day 23 you will see how React Query removes this code.)
3. Answer the Week 2 self-assessment questions at the end of this document.

In this repo

- Read Architecture once from top to bottom. You will not understand everything yet. Highlight the words you do not know and add them to your journal.

Done when

- [ ] I caught up on all unfinished exercises.
- [ ] My RN catalog page shows loading, error, empty and success states.
- [ ] I answered the Week 2 questions.

Day 15 — Flexbox Layout, NativeWind Styling, Theme Tokens and Dark Mode

Goal: Lay out screens with Flexbox and style them with NativeWind and the project's theme.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- **Flexbox** in RN: default `flexDirection` is `column` (not `row` like the web). Key props: `flex: 1`, `justifyContent` (main axis), `alignItems` (cross axis), `gap`, `flexWrap`.
- **NativeWind v4** lets you style RN with **Tailwind CSS** class names: `className="flex-1 items-center p-4"`.
- **Theme tokens:** colors are named by meaning (`bg-background`, `text-foreground`, `bg-primary`, `text-muted-foreground`, `border-border`) and defined as CSS variables for light and dark mode. **Never hard-code colors** like `#ff0000` or `bg-red-500`.
- **Dark mode:** because components use tokens, they switch automatically.
- `cn()` merges class names conditionally.

Read / watch

- React Native — Flexbox
- NativeWind docs
- Repo: Theming

Practice

1. Build a header with a title on the left and an icon button on the right (`flex-row justify-between items-center`).
2. Build a 2-column grid of cards with `flex-row flex-wrap gap-4`.
3. In this repo, open the UI Sandbox and toggle dark mode in Settings. Watch which colors change.
4. Change the `--primary` token in `src/global.css` (light and dark), reload, see every button change. Then undo it.
5. Use `cn()` to give a card a border only when it is "selected".

In this repo

- `src/global.css` — all theme tokens (light + dark). `tailwind.config.js` maps tokens to class names.
- `src/shared/lib/utils.ts` — the `cn()` helper. `src/shared/lib/theme.ts` and `src/shared/hooks/use-theme-colors.ts` — for places that need a raw color value (like icons).
- `src/shared/components/theme-toggle.tsx` — dark mode switch.

Done when

- [ ] I can build row/column layouts with Flexbox from memory.
- [ ] I style only with `className` and theme tokens.
- [ ] I can explain why hard-coded colors break dark mode.

Day 16 — Lists: FlatList, FlashList and Images

Goal: Render long lists that stay smooth on real phones.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- `ScrollView` renders **everything** at once — fine for short content, slow for long lists.
- `FlatList` renders only what is on screen (virtualisation). Key props: `data`, `renderItem`, `keyExtractor`, `ListEmptyComponent`, `ListHeaderComponent`, `onEndReached` (load more), `refreshing` + `onRefresh` (pull-to-refresh).
- **FlashList v2** (by Shopify) is a faster drop-in replacement used in this project.
- Keep list items light; avoid creating new functions/objects for every item when it matters.
- `expo-image` caches images and supports placeholders and transitions.

Read / watch

- FlashList docs
- React Native — Performance overview — the section about lists

Practice

1. In your practice project, show 100 products with `FlatList`, then switch to `FlashList`.
2. Add `ListEmptyComponent` and pull-to-refresh.
3. Add `onEndReached` that logs "load more" to the console.
4. Use `expo-image` for thumbnails with a fixed size and `contentFit="cover"`.

In this repo

- `src/modules/catalog/features/product-list/product-list-screen.tsx` — FlashList with pull-to-refresh, load-more and a debounced search. Find each of these.
- `src/modules/catalog/features/product-list/product-list-skeleton.tsx` — a loading skeleton.
- `src/modules/catalog/components/product-thumbnail.tsx` — `expo-image` usage.

Done when

- [ ] I can explain why `ScrollView` + `map` is bad for long lists.
- [ ] I built a FlashList with empty state, pull-to-refresh and load-more.
- [ ] I found all three behaviours in the product list screen.

Day 17 — Expo Router I: File-Based Routes, Layouts, Stack and Tabs

Goal: Understand how screens and navigation are created from files.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Expo Router** = every file in `src/app/` is a route (a screen with a URL).
- `_layout.tsx` defines the **navigator** for its folder: `Stack` (screens on top of each other, with a back button) or `Tabs` (bottom tabs).
- **Groups** `(tabs)` organise files without adding to the URL.
- `index.tsx` = the default route of a folder. `+not-found.tsx` = the 404 screen.
- `Link` and `router.push()` / `router.back()` for navigation.
- **Thin routes:** in this project route files are tiny; they only render a screen from a module.

Read / watch

- Expo Router — Introduction
- Expo Router — src directory

Practice

1. Draw the route tree of this repo from `src/app/` (root Stack → `(tabs)` → index/products/settings; plus products/new, products/[id], products/[id]/edit, ui-sandbox).
2. Add a new tab "About" in `src/app/(tabs)/` with an icon in `(tabs)/_layout.tsx`. (Undo it afterwards or keep it on a branch.)
3. Add a new Stack screen `src/app/about.tsx` and open it with `router.push('/about')` from Settings.
4. Change the title of a Stack screen in `src/app/_layout.tsx`.

In this repo

- `src/app/_layout.tsx` — root Stack and providers. `src/app/(tabs)/_layout.tsx` — bottom tabs.
- `src/app/(tabs)/products.tsx` — a thin route that renders `ProductListScreen` from `@/modules/catalog`.
- `src/app/ui-sandbox.tsx` — a root Stack screen.

Done when

- [ ] I can say which file creates which screen.
- [ ] I added a tab and a stack screen.
- [ ] I can explain why route files stay thin.

Day 18 — Expo Router II: Dynamic Routes, Params and Typed Routes

Goal: Open a detail screen for a specific item with type-safe navigation.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- **Dynamic route:** `products/[id]/index.tsx` matches `/products/1`, `/products/42` ...
- `useLocalSearchParams<{ id: string }>()` reads params. **Params are always strings** — convert with `Number(id)` and handle invalid values.
- **Typed routes:** Expo Router generates types so `router.push('/prodcts/1')` (typo) is a TypeScript error.
- **Route builders:** functions like `catalogRoutes.detail(id)` keep URLs in one place.
- Passing only the **id** in the URL (not the whole object) and loading data on the detail screen.

Read / watch

- Expo Router — Introduction — dynamic routes section
- Expo Router — Typed routes

Practice

1. Open `src/app/products/[id]/index.tsx` and trace: how is `id` read, converted and passed to `ProductDetailScreen`?
2. Type `/products/abc` as a deep link (or navigate there in code). What happens? Improve the handling if needed.
3. Add a route builder `catalogRoutes.about()` (or similar) and use it instead of a string.
4. Make a typo in a route string and read the TypeScript error.

In this repo

- `src/modules/catalog/catalog.routes.ts` — route builders (`catalogRoutes`).
- `src/app/products/[id]/index.tsx` and `src/app/products/[id]/edit.tsx` — dynamic routes.
- `src/modules/catalog/features/product-detail/product-detail-screen.tsx` — receives the id.

Done when

- [ ] I can create a dynamic route and read its params safely.
- [ ] I use route builders instead of hand-written URL strings.
- [ ] I have seen a typed-route error.

Day 19 — Safe Areas, Keyboard, Platform Differences and UI Primitives

Goal: Make screens that look right on every phone and use the project's UI kit.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- **Safe area** = the part of the screen not covered by the notch, status bar or home indicator. Use `react-native-safe-area-context`.
- **Keyboard handling:** the keyboard can hide inputs. Use keyboard-aware scrolling, `keyboardShouldPersistTaps="handled"`, and dismiss the keyboard on tap outside.
- **Platform differences:** `Platform.OS === 'ios'`, `.ios.tsx` / `.android.tsx` files, different shadows, back button on Android, haptics.
- **React Native Reusables** = copy-in UI primitives (Button, Text, Input, Card, Dialog...) styled with NativeWind. You own the code in `src/shared/ui`.
- Component levels: `shared/ui` (primitives) → `shared/components` (composed) → `modules/<name>/components` or a feature folder.

Read / watch

- React Native Reusables
- Repo: Libraries and Conventions

Practice

1. Open the create-product form on a small phone (or rotate). Does the keyboard hide the submit button? Find how the screen handles it.
2. Add a `Platform.OS` check that shows a different label on iOS and Android.
3. Use `Button`, `Input` and `Card` from `@/shared/ui` to build a "Contact us" card in the UI Sandbox.
4. Add a new primitive with `npx @react-native-reusables/cli@latest add <name> --styling-library nativewind` (for example `switch` or `progress`) and show it in the sandbox.

In this repo

- `src/shared/ui/` — primitives. `src/shared/components/form-field.tsx`, `app-bottom-sheet.tsx`, `floating-action-button.tsx` — composed components.
- `src/shared/lib/haptics.ts` — platform-aware haptics.
- `src/shared/sandbox/` and `src/modules/settings/features/ui-sandbox/` — the UI Sandbox (every shared component must appear there).

Done when

- [ ] My screens respect safe areas and the keyboard never hides an input.
- [ ] I can explain primitive vs composed vs feature components.
- [ ] I added a component to the UI Sandbox.

Phase 4 — Production Data Layer (Days 20–25)

Day 20 — The HTTP Client, Error Model, Endpoints and Environment Variables

Goal: Understand how every request in the app goes through one place and how errors are described.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **One HTTP client:** only `src/shared/api/http-client.ts` may call `fetch`. It adds the base URL, JSON headers, a timeout and turns every failure into an `ApiError`.
- **Error model:** `ApiError` has a `kind`: `network` (no internet), `timeout`, `http` (server answered 4xx/5xx), `parse` (data had the wrong shape). Screens show a friendly message with `getErrorMessage`.
- **Endpoints file:** all URLs of a module live in one file, never scattered in screens.
- **AbortSignal** cancels a request when the screen no longer needs it.
- **Environment variables:** Expo only exposes variables starting with `EXPO_PUBLIC_` to the app. They are **built into the app bundle**, so anyone can read them. **Never put secrets** (private API keys, passwords) there — secrets belong on a server.
- The layers: UI → React Query hook → service → HTTP client → API. Never skip a layer.

Read / watch

- Expo — Environment variables
- Presentation Domain Data Layering (Martin Fowler)
- Repo: Architecture — the layers section

Practice

1. Read `http-client.ts` and write in your journal what happens, step by step, on a 404 and on a timeout.
2. Copy `.env.example` to `.env`, set `EXPO_PUBLIC_API_TIMEOUT_MS=1`, restart with `npx expo start --clear`, and open Products. Which error kind do you see? Undo it.
3. Turn on airplane mode and pull to refresh. Which error kind now?
4. Add a new endpoint to `catalog.endpoints.ts` (for example products by category: `/products/category/<name>`) — data only, no screen yet.
5. Try calling `fetch` directly in a screen and run `npm run lint`. Read the error.

In this repo

- `src/shared/api/http-client.ts`, `api-error.ts`, `parse.ts`, `index.ts`.
- `src/shared/config/env.ts` — reads `EXPO_PUBLIC_API_URL` and `EXPO_PUBLIC_API_TIMEOUT_MS`.
- `src/modules/catalog/data/catalog.endpoints.ts`.

Done when

- [ ] I can list the 4 `ApiError` kinds and when each happens.

- [ ] I can explain why `EXPO_PUBLIC_` variables must never contain secrets.
- [ ] I saw the lint error for calling `fetch` outside the client.

Day 21 — Week 3 Review and Buffer Day

Goal: Make sure React Native, navigation and the request flow are solid before React Query.

Time split (≈4–5 h): Review 2 h · Practice 2 h · Journal & plan 1 h

Learn

- No new concepts. Revisit Days 15–20.

Read / watch

- Repo: Architecture — read it again; it should make much more sense now.
- Repo: Catalog module README

Practice

1. Trace one full request on paper: tap on a product card → route → screen → hook → service → HTTP client → DummyJSON → back to the screen.
2. Build, in your practice project, a two-tab app (Home, List) with a detail screen using a dynamic route, styled with NativeWind.
3. Answer the Week 3 self-assessment questions.

In this repo

- Put a `console.log` in `http-client.ts`, open the detail screen and watch the request in the terminal. Remove it afterwards.

Done when

- [ ] I can trace a request through every layer without looking at the code.
- [ ] My practice app has tabs, a stack and a dynamic route.
- [ ] I answered the Week 3 questions.

Day 22 — zod Validation and the Service Layer

Goal: Never trust API data: validate it, and keep data logic in plain service functions.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- TypeScript types disappear at runtime. If the API sends `price: "12"` instead of `12`, TS cannot catch it. **zod** checks real data at runtime.
- Schema: `const ProductSchema = z.object({ id: z.number(), title: z.string(), brand: z.string().optional() })`.
- One source of truth: `type Product = z.infer<typeof ProductSchema>`.
- `parse` (throws) vs `safeParse` (returns success/error). In this repo, `parseResponse` turns a zod failure into an `ApiError` of kind `parse`.
- **Service** = plain `async` functions (no React, no hooks, no UI). Each one calls the HTTP client with an endpoint and validates the response.
- Form schemas: validation for user input, reused by the form (Day 25).

Read / watch

- zod docs
- DummyJSON — Products docs

Practice

1. Read `product.schema.ts` and match each field with real JSON from `https://dummyjson.com/products/1`.
2. **Break it on purpose:** add a required field that DummyJSON does not send (for example `barcodeNumber: z.number()`) and open a product. See the parse error screen. Then make it `.optional()` and see it work.
3. Add a real field you are not using yet (for example `warrantyInformation` or `shippingInformation`) to the schema and show it on the detail screen.
4. Write a new service function `listByCategory(category)` using the endpoint you added on Day 20, validated with the list schema.
5. Run `npm run typecheck`.

In this repo

- `src/modules/catalog/data/product.schema.ts` — API schemas, inferred types and the form schema.
- `src/modules/catalog/data/products.service.ts` — `productsService.list`, `getById`, `create`, `update`, `remove`, `categories`.
- `src/shared/api/parse.ts` — `parseResponse`.
- `src/modules/catalog/data/local-only-products.ts` — a workaround because DummyJSON does not really save new products (read the comment to understand why).

Done when

- [ ] I can write a zod schema and infer a type from it.
- [ ] I saw (and fixed) a parse error caused by a wrong schema.
- [ ] I wrote a service function that validates its response.

Day 23 — React Query I: Queries, Query Keys, `queryOptions` and Screen States

Goal: Load server data the production way: cached, deduplicated and with clear loading/error/empty states.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Server state** (data that lives on a server) is different from **UI state** (what tab is open). **TanStack React Query** manages server state: caching, refetching, retries, deduplication.
- `useQuery({ queryKey, queryFn })` returns `data`, `isPending`, `isError`, `error`, `refetch`, `isRefetching`.
- **Query keys** are the cache address. Build them with a **factory** (`productKeys.detail(1)`), never by hand. Keys are hierarchical, so you can refresh groups (`productKeys.lists()`).
- **`queryOptions`** puts key + function + options together so the same definition can be used by hooks, prefetching and cache reads.
- `staleTime` (how long data is "fresh") vs `gcTime` (how long unused data stays in memory). Defaults for this app live in `query-client.ts`.
- **Screen states:** every data screen handles **loading**, **error (with retry)**, **empty** and **success**.

Read / watch

- TanStack Query — Overview
- Practical React Query (TkDodo)
- Effective React Query Keys (TkDodo)
- The Query Options API (TkDodo)

Practice

1. Read `product.queries.ts` and `product.keys.ts`. For each hook, write its key and its service call.
2. Open a product, go back, open it again: notice it shows instantly (cache). Explain why in your journal.
3. Add a `byCategory(category)` key to the factory and a `useProductsByCategory(category)` hook using your Day 22 service.
4. Show it on the detail screen as "More from this category", using `Loading`, `Error` and `Empty` from `screen-states.tsx`.
5. Compare with your Day 14 `useEffect` version: which code disappeared?

In this repo

- `src/modules/catalog/data/product.keys.ts` — the key factory.
- `src/modules/catalog/data/product.queries.ts` — `productQueries` (`queryOptions`) and `useProduct`, `useCategories`, `useTopRatedProducts`, `useProductCount`.
- `src/shared/lib/query-client.ts` — global defaults. `src/shared/providers/app-providers.tsx` — where the client is provided.
- `src/shared/components/screen-states.tsx`.

Done when

- [ ] I can explain server state vs UI state.
- [ ] I added a key, a `queryOptions` entry and a hook, and showed all four screen states.
- [ ] I can explain why we never write query keys by hand.

Day 24 — React Query II: Infinite Queries, Mutations, Cache Updates and Optimistic Updates

Goal: Paginate lists and change server data safely.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Infinite queries** (`useInfiniteQuery`): load pages one after another (`limit` / `skip`), with `getNextPageParam` and `fetchNextPage`. Perfect for "load more" in FlashList.
- **Mutations** (`useMutation`): create, update, delete. Callbacks: `onMutate`, `onSuccess`, `onError`, `onSettled`.
- After a mutation, either:
  - **Invalidate** (`invalidateQueries`) — mark data stale and refetch. Simple and always correct.
  - **Update the cache directly** (`setQueryData`) — instant, no extra request, but you must update every place the data appears.
- **Optimistic update:** change the UI **before** the server answers; save a snapshot; on error, **roll back** to the snapshot. Used for delete in this repo.
- Cache helpers keep this logic in one place.

Read / watch

- Mastering Mutations in React Query (TkDodo)
- TanStack Query — Optimistic Updates

Practice

1. Read `useProductList` and find `getNextPageParam`. Scroll the list and watch new pages load.
2. **Add a "sort by price" toggle** to the product list: `productsService.list` already supports `sortBy` and `order`. You must (a) add the sort to the query key filters in `productKeys.list(...)`, (b) pass it to the service, (c) add the toggle button. What happens if you forget step (a)?
3. Read `use-delete-product.ts`: find the snapshot, the optimistic removal and the rollback.
4. Make the delete fail on purpose (for example a wrong endpoint) and watch the item come back.
5. Read `use-create-product.ts` and `use-update-product.ts`: which ones invalidate and which update the cache with `product.cache.ts` helpers?

In this repo

- `src/modules/catalog/data/product.queries.ts` — `useProductList` (infinite).
- `src/modules/catalog/data/product.cache.ts` — cache helpers.
- `src/modules/catalog/features/create-product/use-create-product.ts`, `features/edit-product/use-update-product.ts`, `features/delete-product/use-delete-product.ts`.

Done when

- [ ] My sort toggle works and the sort is part of the query key.
- [ ] I can explain invalidation vs direct cache update and when to pick each.

- [ ] I can explain optimistic update with rollback step by step.

Day 25 — Forms with react-hook-form + zod

Goal: Build forms that validate input, show errors and submit through a mutation.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- **react-hook-form** manages form values, touched fields and errors with very few re-renders.
- `useForm({ resolver: zodResolver(schema), defaultValues })`, `Controller` (needed for React Native inputs), `handleSubmit`, `formState.errors`, `isSubmitting`.
- The same zod form schema validates on the phone; the server may still reject — show server errors too.
- Number inputs: `TextInput` gives strings; convert/coerce with zod.
- UX (User Experience) basics: label every field, show the error under the field, disable the submit button while submitting, keep the keyboard from hiding inputs.
- One form component reused by **create** and **edit** screens.

Read / watch

- react-hook-form — Get Started
- zod docs

Practice

1. Read `product-form.tsx` and find `useForm`, `Controller`, the resolver and the submit handler.
2. Add a validation rule: title must be at least 3 characters; price must be greater than 0. See the errors appear.
3. Add a new optional field (for example `brand`) to the form schema and the form, and send it in create and edit.
4. Make the submit button show a spinner while the mutation is pending.
5. Create a product, then edit it. Check that both screens reuse the same form.

In this repo

- `src/modules/catalog/components/product-form.tsx` — the shared form.
- `src/modules/catalog/components/category-picker-sheet.tsx` — a custom input inside a bottom sheet.
- `src/shared/components/form-field.tsx` — label + input + error.
- `src/modules/catalog/features/create-product/create-product-screen.tsx` and `features/edit-product/edit-product-screen.tsx`.

Done when

- [ ] I can build a form with react-hook-form, `Controller` and a zod resolver.
- [ ] I added a field and a validation rule end to end (schema → form → service).
- [ ] Create and edit both work with my changes.

Phase 5 — Architecture, Quality and Shipping (Days 26–30)

Day 26 — Modular Architecture: Modules, Features and ESLint Boundaries

Goal: Understand why this repo is organised by business area and how the rules keep it clean.

Time split (≈4–5 h): Learn 2 h · Practice 2 h · Build/review 1 h

Learn

- **Vertical slice:** one feature contains everything it needs (screen, hook, components) through all layers, instead of grouping by file type.
- **Module** = a business area (catalog, home, settings) that owns its **data layer** (`data/`) and groups related **features** (`features/product-list` , `features/delete-product` ...).
- **Public API:** other code imports a module only through its `index.ts` (`@/modules/catalog`). Everything else is private.
- **Colocation:** keep code close to where it is used. Move it to `shared/` only when two modules need it.
- **ESLint boundary rules** in this repo:
  - import modules only via `@/modules/<name>` ;
  - `shared` cannot import from `modules` ;
  - features import sibling features only through their `index.ts` ;
  - UI cannot import services or the HTTP client;
  - only the HTTP client may call `fetch` .

Read / watch

- Vertical Slice Architecture (Jimmy Bogard)
- Modular Monolith primer (Kamil Grzybek)
- Colocation (Kent C. Dodds)
- Bulletproof React — project structure
- Feature-Sliced Design overview (optional, a related approach)
- ESLint — no-restricted-imports
- Expo — Using ESLint
- Repo: Architecture, Adding a module, Adding a feature, Catalog module README, Product list feature README

Practice

1. Draw the folder tree of `src/modules/catalog` and label each folder: public API, routes, data, components, features.
2. **Break each boundary rule once** and run `npm run lint` : import `@/modules/catalog/data/products.service` from `src/app` ; import a module from `src/shared` ; import `productsService` inside a screen. Read every error, then undo.
3. Read how `home-screen.tsx` uses only catalog's public API (`TopRatedCarousel` , `useProductCount`).
4. Following Adding a feature, add a small feature to catalog: `features/category-products/` showing products of one category (reuse your Day 23 hook), exported through its `index.ts` and catalog's `index.ts` , with a route in `src/app` .

5. Run `npm run validate`.

In this repo

- `eslint.config.js` — the boundary rules (read the comments).
- `src/modules/catalog/index.ts`, `catalog.routes.ts`, `features/*/index.ts`.
- `src/modules/home/features/dashboard/home-screen.tsx` — a module using another module's public API.

Done when

- [ ] I can explain module vs feature vs shared, and why imports go through `index.ts`.
- [ ] I triggered and understood each boundary lint error.
- [ ] I added a feature and `npm run validate` passes.

Day 27 — Testing with Jest + Conventional Commits and Pull Requests

Goal: Prove your data layer works with tests, and deliver work the way teams do.

Time split (≈4–5 h): Learn 1.5 h · Practice 2.5 h · Build/review 1 h

Learn

- **Unit test** = checks one small piece (a function) in isolation. **Jest** runs tests: `describe`, `it` / `test`, `expect(...).toBe/ toEqual/ rejects.toThrow`.
- **Arrange** → **Act** → **Assert** structure.
- **Mocking:** replace `fetch` (or the HTTP client) with a fake that returns what you want, so tests do not need the internet.
- What to test first in this architecture: **schemas** (valid and invalid data) and **services** (correct endpoint, validation, error mapping). Component tests with React Native Testing Library come later.
- **Conventional Commits:** `feat: add sort toggle`, `fix: handle 404 on detail`, `test: cover getById errors`, `docs:`, `refactor:`, `chore:`.
- **PR workflow:** branch → small commits → `npm run validate` → push → open PR with the template → CI (Continuous Integration) runs → review → merge.

Read / watch

- Jest — Getting Started
- Expo — Unit testing
- React Native Testing Library (skim, for later)
- Conventional Commits

Practice

1. Run `npm test`. Then run `npm run test:watch` and change a test to see it fail.
2. Read `product.schema.test.ts` and `products.service.test.ts`. Write down how `fetch` is mocked.
3. **Write a test for `productsService.getById` when the server answers 404:** it should reject with an `ApiError` whose kind is `http`.
4. Write a test that `getById` rejects with kind `parse` when the response has the wrong shape.
5. Write a schema test for the field you added on Day 22.
6. Write a test for your `listByCategory` service (correct URL, validated data).
7. Create a branch, commit with Conventional Commit messages, push, and open a PR in **your fork** using `.github/pull_request_template.md`. Watch CI run.

In this repo

- `jest.config.js`, `src/modules/catalog/data/__tests__/`.
- `.github/workflows/ci.yml` — what CI runs on every PR. `.github/pull_request_template.md`.

Done when

- [ ] I wrote at least 4 new passing tests, including the `getById` 404 test.
- [ ] My commits follow Conventional Commits.
- [ ] I opened a PR and CI passed.

Day 28 — Debugging, Performance, Accessibility + Week 4 Review

Goal: Find bugs quickly, keep the app fast and usable for everyone, and review the week.

Time split (≈4–5 h): Learn 1.5 h · Practice 2 h · Review 1.5 h

Learn

- **Debugging tools:** logs in the terminal, the dev menu (shake the phone or press `m` in the terminal), React Native DevTools (press `j`), breakpoints, inspecting network requests, React Query devtools ideas.
- Read the **red screen** (error) and **yellow box** (warning) carefully; the first line of your own code in the stack trace is usually the cause.
- **Performance basics:** test on a real (and older) phone; use FlashList for lists; avoid unnecessary re-renders; keep heavy work off the render path; optimise images; measure before optimising.
- **Accessibility (a11y) basics:** `accessibilityLabel` for icon-only buttons, `accessibilityRole`, touch targets at least 44×44 points, good color contrast (theme tokens help), support large font sizes, test with VoiceOver (iOS) / TalkBack (Android).

Read / watch

- Expo — Debugging tools
- React Native — Performance overview

Practice

1. Open React Native DevTools, set a breakpoint in `productsService.getById`, open a product and step through.
2. Introduce a bug (rename a field in a component), read the red screen, find the line, fix it.
3. Turn on VoiceOver/TalkBack and navigate the product list. Add an `accessibilityLabel` to every icon-only button you find without one.
4. Increase the phone's font size to the maximum. Fix one layout that breaks.
5. **Review:** answer the Week 4 self-assessment questions and redo any exercise from Days 22–27 that you skipped.

In this repo

- `src/modules/catalog/features/product-list/product-actions-sheet.tsx` and `src/shared/components/floating-action-button.tsx` — check their accessibility labels.
- `src/modules/catalog/features/product-list/product-list-screen.tsx` — look for performance choices (FlashList, debounced search).

Done when

- [ ] I debugged with a breakpoint, not only `console.log`.
- [ ] I fixed at least 2 accessibility issues.
- [ ] I answered the Week 4 questions.

Day 29 — Development Builds, EAS Build / Submit / Update + Capstone Part 1

Goal: Understand how an app gets from your laptop to the stores, then start the capstone.

Time split (≈4–5 h): Learn 1.5 h · Capstone 3 h · Journal 0.5 h

Learn

- **Expo Go vs development build:** Expo Go only contains Expo's bundled native modules. As soon as you add a library with custom native code, you need a **development build** (your own version of "Expo Go" for your app).
- `ios/` and `android/` folders are **generated** (CNG — Continuous Native Generation). Never edit them by hand; configure native settings in `app.json` and config plugins.
- **EAS (Expo Application Services):**
  - **EAS Build** builds signed app binaries in the cloud (`npx eas-cli@latest build`), with profiles in `eas.json` (development, preview, production).
  - **EAS Submit** uploads builds to the App Store / Google Play.
  - **EAS Update** ships JavaScript-only fixes **over the air** (OTA) without a store review. Native changes still need a new build.
- `npm run doctor` (expo-doctor) checks dependency and config problems.

Read / watch

- Expo — Development builds
- EAS Build — Introduction
- EAS Update — Introduction
- Repo: Adding a module

Practice

1. Run `npm run doctor` and read the result.
2. Read `app.json`. Find the app name, slug, scheme and plugins.
3. (Optional, needs an Expo account) Run `npx eas-cli@latest build --profile development --platform android` and install the build.
4. **Capstone part 1 — the "users" module data layer.** Create a branch `feat/users-module` and follow Adding a module:
   - `src/modules/users/data/users.endpoints.ts` — `/users`, `/users/{id}`, `/users/search?q=`.
   - `user.schema.ts` — zod schemas for a user (id, firstName, lastName, email, phone, image, age, optional company) and the list response (`users`, `total`, `skip`, `limit`). Check real JSON at `https://dummyjson.com/users/1`.
   - `users.service.ts` — `list`, `getById`, validated with `parseResponse`.
   - `user.keys.ts` and `user.queries.ts` — key factory, `queryOptions`, `useUserList` (infinite) and `useUser`.
   - `__tests__/` — schema tests and a service test (including a 404).
   - `index.ts` — public API (empty for now apart from types).

5. Run `npm run validate` and commit with Conventional Commits.

In this repo

- `app.json`, `eas.json` (if present), `package.json` → `doctor` script.
- Use `src/modules/catalog/data/` as your reference for every file.

Done when

- [ ] I can explain Expo Go vs development build, and EAS Build vs EAS Update.
- [ ] The users data layer exists with passing tests.
- [ ] `npm run validate` passes on my branch.

Day 30 — Capstone Part 2: Finish the "users" Module and Open a Pull Request

Goal: Ship a complete, reviewed feature module that follows every rule of this project.

Time split (≈4–5 h): Build 3.5 h · Review & PR 1 h · Celebrate 0.5 h

Learn

- No new concepts. Today you prove you can combine everything.
- Definition of done for production code: works, typed, validated, tested, linted, accessible, all screen states handled, documented, reviewed.

Read / watch

- Repo: Adding a feature, Conventions, Product list feature README

Practice

1. **Feature** `user-list` (`src/modules/users/features/user-list/`): FlashList of users with avatar (`expo-image`), name and email; infinite scroll; pull-to-refresh; debounced search using `/users/search`; loading skeleton, error with retry and empty state.
2. **Feature** `user-detail` (`src/modules/users/features/user-detail/`): avatar, full name, email, phone, age, company; all screen states.
3. **Routes:** `users.routes.ts` with route builders; thin route files — a new tab `src/app/(tabs)/users.tsx` and a stack screen `src/app/users/[id].tsx` that reads `id` with `useLocalSearchParams`.
4. Export only what `src/app` needs from `src/modules/users/index.ts`.
5. Add a `README.md` for the module and the list feature, copying the style of the catalog ones.
6. **Stretch:** show a "Top users" widget on the Home dashboard through the users public API; add a sandbox section for any new shared component.
7. Run `npm run validate`. Fix everything.
8. Push and open a PR using the template. Title: `feat(users): add users module with list and detail`. Ask your mentor for a review and fix the comments.

In this repo

- `src/modules/users/` (your new module), `src/app/(tabs)/_layout.tsx` (new tab), `src/app/users/[id].tsx`.
- Compare your work file by file with `src/modules/catalog/`.

Done when

- [ ] The Users tab works on my phone: list, search, infinite scroll, refresh, detail.
- [ ] Every screen handles loading, error and empty states.
- [ ] No boundary rule is broken and `npm run validate` passes.
- [ ] My PR is open, CI is green, and I responded to review comments.

Weekly self-assessment questions

Answer these in your journal without looking anything up. If you cannot answer one, that is your review topic.

Week 1 (Days 1–7)

1. What happens, step by step, when an app requests `GET /products/1`?
2. What is the difference between `git commit` and `git push`? Between a branch and a pull request?
3. When do you use `const`, `let`, `??` and `?.`?
4. Write `map`, `filter` and `reduce` examples from memory.
5. Why must you check `res.ok` after `fetch`?
6. What is a closure? Give an example.
7. What is the difference between `unknown` and `any`? How do you narrow `unknown`?
8. What do `Partial`, `Pick`, `Omit` and `as const` do?

Week 2 (Days 8–14)

1. What are props, and why are they read-only?
2. Why must you never change state directly?
3. Why do list items need stable keys? Why not the index?
4. Name three situations where you should not use `useEffect`.
5. What are the rules of hooks?
6. What does "lifting state up" mean?
7. What is the difference between Expo, Expo Go and React Native?

Week 3 (Days 15–21)

1. Why does RN Flexbox default to `column`? What do `justifyContent` and `alignItems` control?
2. Why should you use theme tokens instead of hard-coded colors?
3. Why is FlashList better than `ScrollView` + `map` for long lists?
4. Which file creates the bottom tabs? Which creates the root stack?
5. Why are route params always strings, and what must you do about it?
6. What are the four `ApiError` kinds?
7. Why must `EXPO_PUBLIC_` variables never contain secrets?

Week 4 (Days 22–28)

1. Why do we validate API responses with zod if we already use TypeScript?
2. What belongs in a service, and what must never be in a service?
3. What is a query key, and why do we use a key factory?
4. What is the difference between invalidating a query and updating the cache?

5. Explain an optimistic delete with rollback.
6. Why does a React Native form need `Controller`?
7. What is the difference between a module and a feature? What is a module's public API?
8. What would you unit-test first in a new module?
9. Name three accessibility checks every screen should pass.

What comes after Day 30

You now know the core of production React Native. On the job you will meet these topics next. Learn them when a real task needs them:

- **Authentication flows:** sign in / sign out, protected routes and redirects with Expo Router. Start with Expo Router — Authentication.
- **Secure storage:** store tokens with expo-secure-store, never in plain storage.
- **Offline storage and caching:** saving data on the device so the app works without internet. Start with Expo — Store data.
- **Push notifications:** permissions, tokens, sending and handling notifications.
- **Animations and gestures:** React Native Reanimated.
- **Analytics and crash reporting:** understand how users use the app and catch crashes in production.
- **Component tests and end-to-end (E2E) tests:** React Native Testing Library and E2E tools.
- **CI/CD (Continuous Integration / Continuous Delivery):** automated checks, builds and releases on every merge.
- **App store release process:** developer accounts, signing, screenshots, privacy forms, review guidelines, staged rollouts, and versioning with EAS Build and EAS Update.

Glossary of key terms

Term	Meaning
API (Application Programming Interface)	A set of URLs a server offers so apps can read and change data.
ApiError	This project's error class; its `kind` is `network`, `timeout`, `http` or `parse`.
Async / await	Syntax for waiting for a Promise without blocking the app.
Branch	A separate line of work in Git.
Cache	Saved copies of data so it can be shown again quickly. React Query manages it.
CI (Continuous Integration)	Automated checks (typecheck, lint, tests) that run on every pull request.
Closure	A function that remembers variables from where it was created.
CNG (Continuous Native Generation)	Expo generates the `ios/` and `android/` folders from `app.json`; do not edit them by hand.

Term	Meaning
Component	A function that returns UI.
Conventional Commits	A commit message format: `type(scope): description`.
Development build	Your own debug version of the app, needed when you use custom native code.
EAS (Expo Application Services)	Expo's cloud services: Build, Submit, Update.
Endpoint	One URL of an API, e.g. `/products/1`.
ESLint	A tool that finds problems and enforces rules (including this project's boundary rules).
Expo Go	An app that runs your project during development without building it.
Expo Router	File-based navigation: files in `src/app` become screens.
Feature	A vertical slice inside a module (screen + hooks + components for one user task).
FlashList	A fast, virtualised list component by Shopify.
Hook	A function starting with `use` that lets components use React features.
HTTP (HyperText Transfer Protocol)	The protocol clients and servers use to talk.
Infinite query	A React Query query that loads data page by page.
Invalidation	Marking cached data as stale so React Query refetches it.
JSON (JavaScript Object Notation)	A text format for data.
JSX (JavaScript XML)	HTML-like syntax for describing UI in JavaScript.
Module	A business area (e.g. catalog) that owns its data layer and groups related features.
Mutation	A request that changes server data (create, update, delete).
NativeWind	Tailwind CSS class names for React Native (`className`).
OTA (Over The Air) update	A JavaScript update delivered with EAS Update, without a store release.
Optimistic update	Updating the UI before the server confirms, with rollback on error.
Props	Inputs passed to a component.
Public API (of a module)	The module's `index.ts`: the only file other code may import from.
Pull request (PR)	A request to merge your branch, where code is reviewed.
Query key	The unique address of cached data in React Query.
React Query (TanStack Query)	A library for fetching, caching and updating server data.
Safe area	The screen area not covered by notches, status bar or home indicator.

Term	Meaning
Schema (zod)	A runtime description of data shape used to validate it.
SDK (Software Development Kit)	A versioned set of tools and libraries (e.g. Expo SDK 58).
Service	Plain async functions that call the HTTP client and validate responses.
State	Data a component remembers between renders.
Theme token	A named color (e.g. `primary`, `background`) defined for light and dark mode.
TypeScript	JavaScript with types, checked before the app runs.
Vertical slice	Organising code by feature through all layers instead of by file type.

Core concepts you must know before a production app

Tick each item honestly. Everything here was covered in the 30 days.

Foundations

- [ ] I can explain client, server, HTTP methods, status codes and JSON.
- [ ] I use Git branches, small commits and pull requests every day.
- [ ] I am comfortable with modern JavaScript: destructuring, spread, array methods, modules, async/await, try/catch.
- [ ] I write TypeScript without `any`, and I can read generics and utility types.

React and React Native

- [ ] I build small typed components with props and composition.
- [ ] I manage state immutably and know where each piece of state lives.
- [ ] I know when NOT to use `useEffect`.
- [ ] I follow the rules of hooks and can write custom hooks.
- [ ] I lay out screens with Flexbox and style them with NativeWind theme tokens (no hard-coded colors).
- [ ] I use FlashList for long lists.
- [ ] I use Expo Router: layouts, Stack, Tabs, dynamic routes, typed routes and route builders.
- [ ] My screens handle safe areas, the keyboard and platform differences.

Data layer

- [ ] All requests go through one HTTP client; only it calls `fetch`.
- [ ] Every API response is validated with zod in the service layer.
- [ ] I use React Query for server state, with query key factories and `queryOptions`.
- [ ] I can implement pagination, mutations, invalidation and optimistic updates with rollback.
- [ ] Every data screen shows loading, error (with retry), empty and success states.
- [ ] I build forms with react-hook-form + zod.
- [ ] I never put secrets in `EXPO_PUBLIC_` environment variables.

Architecture, quality and shipping

- [ ] I organise code into modules and features and import modules only through their public API.
- [ ] I understand and respect the ESLint boundary rules.
- [ ] I unit-test schemas and services with Jest, mocking the network.
- [ ] I run `npm run validate` before every pull request.
- [ ] I debug with DevTools and breakpoints, and I check accessibility labels, roles and font scaling.
- [ ] I know when I need a development build and how EAS Build, Submit and Update fit together.
- [ ] I write Conventional Commits and clear pull request descriptions.

Congratulations on finishing the plan. Keep your journal going — the habit of learning a little every day is what turns a beginner into a professional.